

ESM: A Framework for Building Effective Surrogate Models for Hardware-Aware Neural Architecture Search

Azaz-Ur-Rehman Nasir*, Samroz Ahmad Shoaib*, Muhammad Abdullah Hanif*, Muhammad Shafique
eBRAIN Lab, Division of Engineering, New York University Abu Dhabi, UAE
{an3525, ss14256, mh6117, muhammad.shafique}@nyu.edu

Abstract—Hardware-aware Neural Architecture Search (NAS) is one of the most promising techniques for designing efficient Deep Neural Networks (DNNs) for resource-constrained devices. Surrogate models play a crucial role in hardware-aware NAS as they enable efficient prediction of performance characteristics (e.g., inference latency and energy consumption) of different candidate models on the target hardware device. In this paper, we focus on building hardware-aware latency prediction models. We study different types of surrogate models and highlight their strengths and weaknesses. We perform a systematic analysis to understand the impact of different factors that can influence the prediction accuracy of these models, aiming to assess the importance of each stage involved in the model designing process and identify methods and policies necessary for designing/training an effective estimation model, specifically for GPU-powered devices. Based on the insights gained from the analysis, we present a holistic framework that enables reliable dataset generation and efficient model generation, considering the overall costs of different stages of the model generation pipeline.

Index Terms—Neural Architecture Search, Surrogate Models, Hardware, Latency Prediction, Resource-Constrained Devices

I. INTRODUCTION AND MOTIVATION

Deep Neural Networks (DNNs) are nowadays being used to serve a wide variety of tasks in different domains such as computer vision [1] and Natural Language Processing (NLP) [2] [3]. Moreover, the diverse set of devices used for DNN deployment in real-world application (e.g., embedded CPU and GPU devices, as well as specialized hardware accelerators [4]) brings a unique challenge, as DNNs result in different latency and energy numbers on different devices because of the differences in their hardware architecture and capabilities [5]. Neural Architecture Search (NAS), specifically, hardware-aware NAS (HW-NAS), techniques have been proposed to automatically search for DNNs based on the user-defined performance and hardware constraints [6]. A generic overview of hardware-aware NAS techniques is presented in Fig. 1.

Various HW-NAS techniques have been proposed in the literature. One prominent set of techniques focuses on simultaneous exploration and training of candidate models to converge to a suitable solution. In contrast, methods like Once-for-All (OFA) [5] adopt a decoupled strategy, where a supernet is first trained, followed by exploration of DNN configurations based on user-defined constraints, without involving additional training. The core benefit of the latter category lies in fast exploration, as it eliminates model training iterations. Building on this advantage, our work focuses on improving the exploration phase of OFA-based techniques to efficiently and accurately identify suitable models for deployment on target hardware, while satisfying all user-defined performance constraints.

Since deploying each candidate model on hardware to evaluate its performance is time-consuming, HW-NAS techniques

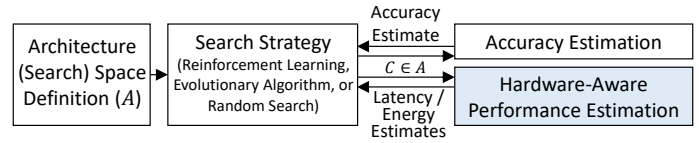


Fig. 1. A general overview of hardware-aware NAS which highlights the importance of hardware-aware performance (latency or energy) estimation for guiding the architecture search strategy.

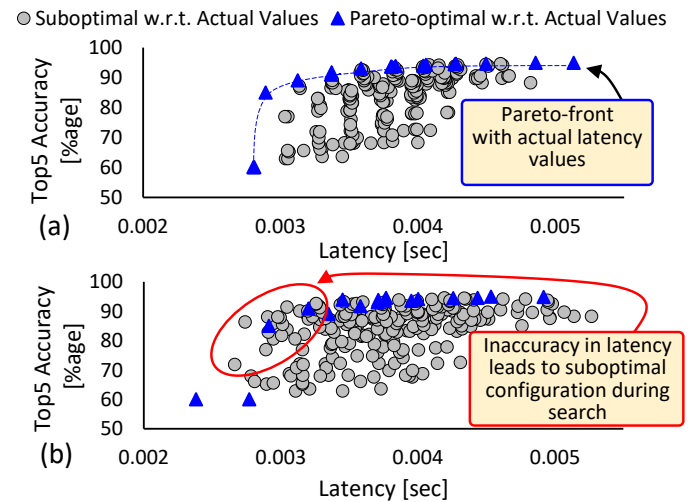


Fig. 2. Impact of inaccuracy in latency prediction on neural architecture search results. (a) illustrates the top-5 accuracy vs. latency plot of 243 different ResNet architectures generated by varying the depth values of the pre-trained ResNet50 supernet made available by Cai et al. [5]. The accuracy numbers are generated by evaluating each candidate model on the ImageNet validation set, and the latency numbers are generated by running the model on an RTX4090 GPU. (b) illustrates how the actual Pareto-optimal points can potentially move in the architecture space when the latency prediction is slightly inaccurate.

utilize surrogate models to efficiently estimate the performance or latency of DNNs on the target hardware [7] [8] [9] [10] [11]. Fig. 2 highlights the importance of using a precise latency prediction model to converge to models that lie on or near the actual Pareto front. Various techniques have been proposed for building surrogate models for HW-NAS. Earlier methods relied on proxy metrics (e.g., FLOPs) to estimate DNN performance [12] [13]. However, such metrics are hardware-agnostic and fail to accurately reflect a model's true performance or latency on the target hardware. To address this limitation, lookup table and ML-based techniques have been proposed for latency prediction. Lookup table-based techniques use an additive model where latency of each individual layer is taken from the lookup table (defined through profiling) and then the latencies of all the layers are accumulated to produce the latency of the complete model. These techniques result in suboptimal results as they lack in capturing the complex interactions between layers. The ML-based techniques encode the model architecture into an intermediate representation, which serves as input for performance predictors. The most common architecture encod-

*These authors contributed equally to this work

ing types include one-hot [14] [15], and feature representation encodings [16] [17] [18]. These are used with models such as multilayer perceptron (MLP), linear regression, decision tree regression, and boosted decision tree regression [15] [18] [16] [17]. Other techniques leverage the graphical structure of DNNs to utilize Graph Convolutional Networks (GCNs) for building performance predictors [14] [19]. However, these ML models lack in providing accurate latency estimates as will be demonstrated in the following motivational case study.

A. Motivational Case Study

In this section, we highlight the impact of increasing the training set size on the accuracy of latency prediction models. For this analysis, we considered two different supernet architecture types: ResNet and DenseNet architectures. The configurations for both are presented in the experimental setup section (III.A). Note that the architecture space considered for ResNet supernet is much larger than that considered for DenseNet. For prediction model, we used MLP together with summary feature representation encoding from [11]. For each architecture type, we randomly sampled 24000 samples from the architecture space and executed them on an RTX4090 machine to gather architecture-latency pairs for predictor training and evaluation. We used 4000 samples for testing and 8000 or 20000 for training. Fig. 3(e) shows that, for both the architecture types, increasing the training set size from 8000 architecture samples to 20000 samples does not result in any significant improvement in the prediction accuracy. Moreover, as the architecture space considered for DenseNet supernet is much smaller than that of ResNet, the predictor results in significantly higher accuracy for DenseNet. We argue that the lower accuracy in the case of ResNet model is due to the overlapping representation of configurations in the representation space, and thus, *there is a need to explore alternative encoding schemes that can result in better accuracy for larger OFA architecture spaces.*

In addition to the above, we compare the time required to accurately compute the latency of a sample architecture on an RTX4090 GPU with the time needed to train a latency predictor. Fig.4(a) shows that the average latency measurement time per model is comparable to the time required to train a latency predictor with over 8,000 samples. Note that each latency measurement involves multiple inferences to account for fluctuations, as illustrated in Fig.4(b). *On one hand, this emphasizes the high cost of data acquisition, while on the other hand, it highlights the potential for an iterative methodology—comprising training, testing, and dataset extension if necessary—to ensure early exit and minimal overall model generation cost.*

B. Our Novel Contributions

To address the above challenges, in this paper, we make the following key contributions.

- 1) We propose ESM, a framework for building effective latency prediction models for HW-NAS algorithms. ESM targets layer/block-wise architecture spaces built on a fixed macro-architecture. The framework adopts a *train-evaluate-extend* approach, allowing it to efficiently converge on an accurate model that meets user-defined accuracy constraints while minimizing dataset collection costs.

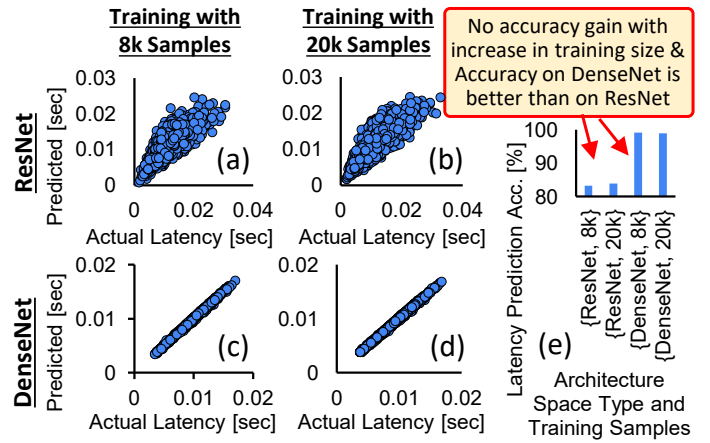


Fig. 3. Actual vs. predicted latency for 4k ResNet model configurations using (a) a predictor trained with 8,000 samples and (b) a predictor trained with 20k samples. (c) and (d) present the results for DenseNet configurations. (e) Comparison of the average accuracy of predictors trained with 8,000 samples vs. those trained with 20,000 samples for the two model types.

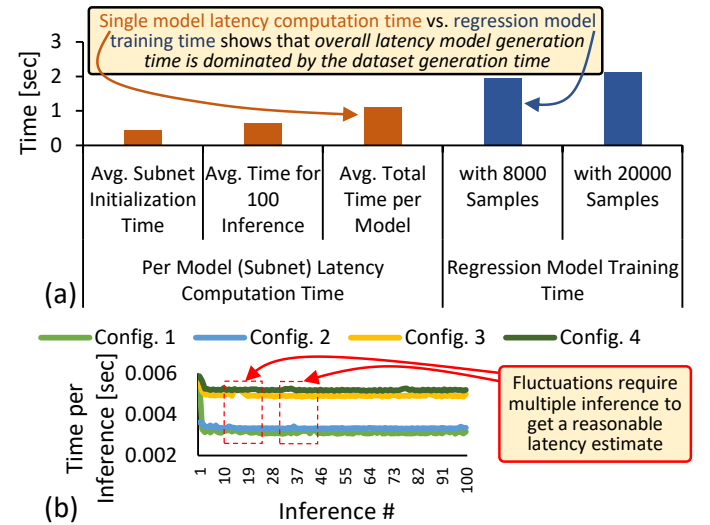


Fig. 4. (a) Comparison between the time required to accurately compute the latency of a model derived from the pre-trained ResNet50 supernet by Cai et al. [5] on an RTX4090 GPU and the training time for a latency predictor. On average, training a regression model of reasonable size on an RTX4090 GPU takes approximately the same amount of time as computing the latency of a model on the target hardware. (b) Latency values across inferences for different architectural configurations, illustrating the need for multiple iterations to obtain a reliable value.

- 2) We propose multiple different architecture encoding schemes specialized for building accurate latency prediction models for layer/block-wise operation space built on a fixed macro-architecture. Our encoding schemes provide a balance between accuracy and surrogate model complexity, without significantly increasing the need for larger training datasets to achieve high accuracy. This is achieved by enhancing the representation capability of the encoding while minimizing the risk of increased sparsity, which could otherwise require larger datasets.
- 3) Our results on a variety of models and devices demonstrate the effectiveness of our proposed encoding schemes over state-of-the-art encoding schemes, specifically for highly diverse architecture search spaces based on fixed macro-architectures.

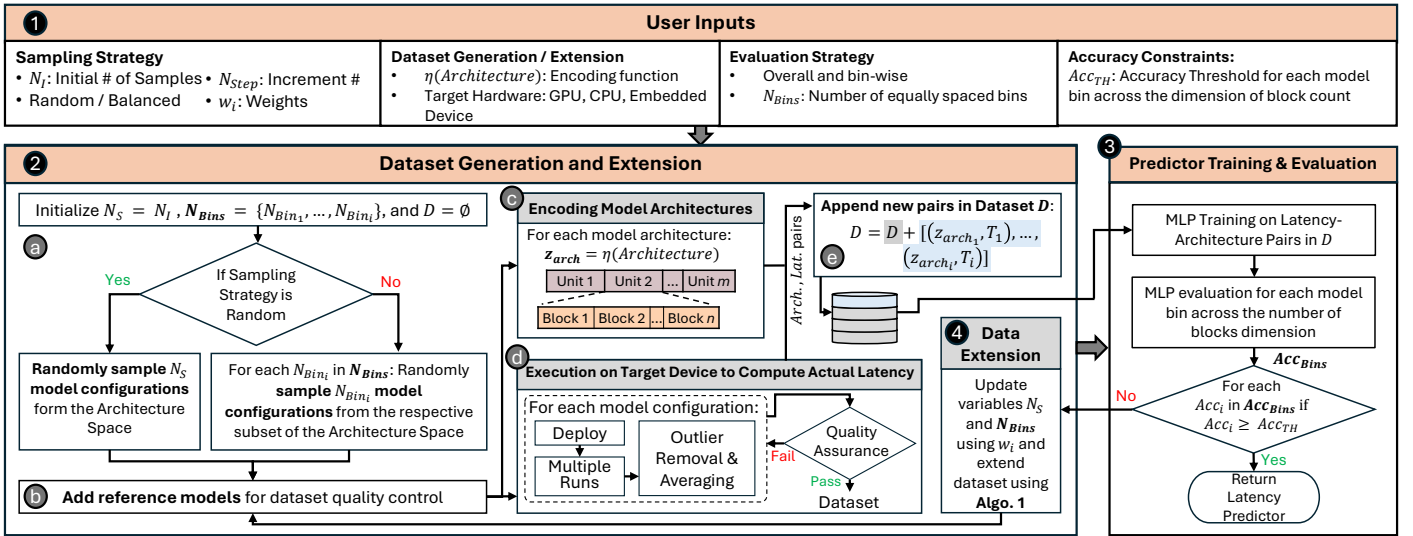


Fig. 5. Overview of our ESM Framework.

II. METHODOLOGY

A. Overview

Fig. 5 provides an overview of our novel framework that can produce an accurate latency predictor. The user defines number of initial samples, sampling strategy, encoding type, evaluation strategy, and accuracy constraints. Based on the selected sampling strategy, model configurations are sampled from the Architecture Space. The selected architectures are encoded, and also have their inference performed on the target hardware. These architecture-latency pairs make up the dataset. An MLP-based latency predictor is trained on this dataset, and the performance of the predictor is evaluated. If the user-defined accuracy constraints are met, the framework returns the latency predictor. Otherwise, the dataset is extended and the predictor is retrained until the accuracy constraints are met.

B. User Inputs

The user defines the following inputs for the framework (see 1 in Fig. 5):

- 1) **Sampling strategy:** *random* or *balanced*.
- 2) **Target Hardware:** GPU, CPU or an embedded device.
- 3) N_I : Number of initial samples to be sampled from the architecture space.
- 4) N_{Step} : Number of additional samples that will be added to N_I during dataset extension.
- 5) w_i : Weights assigned to below accuracy and above accuracy threshold bins.
- 6) $\eta(\text{Architecture})$: Architecture encoding function
- 7) **Evaluation Strategy:** Overall or bin-wise accuracy
- 8) N_{Bins} : Number of equally spaced bins along number of blocks (depth) dimension on which the dataset will be divided
- 9) Acc_{TH} : Accuracy threshold for N_{Bins} to meet to pass the evaluation strategy

C. Dataset Generation

Once the user has defined their inputs, the framework proceeds based on the selected sampling (*random* or *balanced*)

strategy (see a in Fig. 5). Moreover, based on the number of blocks in each model, the dataset is divided into equally spaced N_{Bins} bins.

- 1) **Random Sampling:** In this sampling strategy, we randomly select a given number of models from the entire architectural space. Fig 7 outlines the general structure of our models. Each model is made up of k units; a unit is composed of l number of blocks. For each block, user defines features that they want to vary. For example, in our experiments we fixed each ResNet model to have 4 units, and each unit could have 1 to 7 blocks. When randomly selecting for number of blocks of each unit, a Gaussian spread between the least and the most number of blocks is seen due to the central limit theorem. This bias in the sampling distribution forces the predictor to perform poorly for models present in the corner cases.
- 2) **Balanced Sampling:** To counter the bias that arises naturally with random sampling, the user can select *balanced* strategy. The entire architecture space is divided into equally spaced bins, N_{Bins} , based on the number of blocks. For each bin in N_{Bins} , equal number of models are sampled from the architecture space. This prevents the under-representation of the corner cases and allows the predictor to effectively learn their trends as well. Each block's kernel size and width expansion ratio is still randomly selected. Once models have been sampled from the architecture space, reference models (randomly selected from the architecture space) are added to the sample space. These reference models will be later used for dataset quality control.
- 3) **Execution on Target Device and Latency Computation:** On the user-defined target hardware, inference is performed (see d in Fig. 5). For every sampled model, we record the latency 150 times. The slowest and the fastest 20% of the inferences are discarded, and average is computed of the remaining 60%. As a quality control measure for our dataset, the latencies of the reference models are analyzed. Using reference models, we ensure the quality

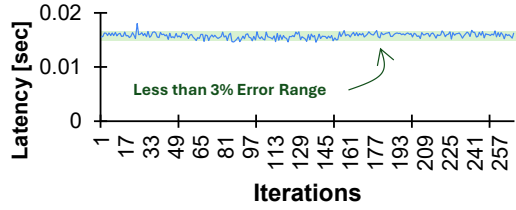


Fig. 6. Variance Plot of Reference models. Most of reference model instances fall within the 3% error boundary. The outliers are removed to improve the dataset quality.

of the dataset. This enables us to record the variance of our dataset, and identify outliers. If the variance is less than 3% (Fig. 6), the dataset passes the quality assurance; otherwise, it fails and the execution is performed again.

- 4) **Encoding Sampled Model Architectures:** Architectures of DNNs can be represented in various forms. Model architecture configurations after ⑤ in Fig. 5, get encoded into vectors (see ③ in Fig. 5). Let's say that a specific encoding scheme (η) is a function that translates an architecture to a vector.

$$\mathbf{z}_{\text{arch}} = \eta(\text{Architecture})$$

These representations depend largely on the type of architectural space the user defines. Typically, one-hot vectors can be utilized to reflect any DNN. But such a representation gives rise to vectors that are extremely lengthy and sparse. Several state-of-the-art techniques employ feature encoding to shorten and reduce the sparsity of the vectors while aiming to maintain feature information [16] [17] [18]. Another common technique is to take mean and standard deviation of features [11] (Fig. 7d). Although both of these embeddings significantly shorten the vector as compared to one-hot vectors, they do not completely solve the problem. Statistical encoding (Fig. 7d) make the vectors too dense, causing loss of crucial information, and disabling the predictor to efficiently learn all parameters. On the other hand, Feature encoding (Fig. 7d) does detail out features of an architecture, the vector is still quite lengthy and sparse.

We propose an encoding scheme, Feature Combination Count (FCC) Encoding, that is not only significantly shorter than one-hot vectors, but also is not too dense (Fig. 7c). FCC first makes combinations of all possible features of a block, and then counts the number of instances each combination is used in the given model. This allows the vectors to retain enough information about any specific model's features for the predictor to perform optimally, while also getting rid of sparsity. We also experimented with Feature Count Encoding (Fig. 7c). This embedding counts the number of occurrences of each feature. Granted, this encoding retained a healthy amount of feature information; however, the vectors were still lengthy and sparse.

The dataset D is made up of these architecture-latency pairs obtained (Fig. 5e).

D. Predictor Training and Evaluation

The latency predictor is an MLP-based regression model designed to predict the latency of any given model architecture within a specific sample space for the target hardware. Primarily, we train our MLP on dataset D and perform evaluation on

each of the bins (see ③ in Fig. 5). This evaluation is based on the accuracy threshold, Acc_{TH} , defined by the user. If all the bin's accuracy, Acc_i , is greater than the threshold, Acc_{TH} , the predictor meets the evaluation criteria. If not, all the accuracies of the bins, Acc_{Bins} , are noted. The framework proceeds to dataset extension, after which the predictor is retrained and re-evaluated.

Algorithm 1: Pseudo-code for Dataset Extension

Input: SS : Entire Sample Space
Input: SS_I : Initial Sampling Space
Input: N_{Step} : Total number of additional training samples to select
Input: N_I : Number of Initial Samples
Input: Acc_{TH} : Accuracy threshold for bin categorization
Input: strategy: Sampling strategy, “random” or “balanced”
Input: w_1 = Weight of Below Threshold Bins
Input: w_2 = Weight of Above Threshold Bins
Output: Randomly sampled additional data $SS_{I,rand}$
Output: Weighted sampled additional data $SS_{I,weighted}$

```

1 if strategy = random then
2   Randomly select  $N_{Step}$  samples from  $SS$  to form  $SS_{rand}$ ;
3   Append  $SS_{rand}$  to  $SS_I$ 
4    $SS_{I,rand} \leftarrow SS_I + SS_{rand}$ 
5   return  $SS_{I,rand}$ 
6 end
7 else if strategy = balanced then
8   Initialize empty list of No. of samples per bin:  $\mathbf{N}_{Bins}$ 
9   /* Separate bins  $N_{Bin_i}$  into two groups */
10   $Below_{Acc_{TH}} \leftarrow$  Bins with accuracy  $< Acc_{TH}$ 
11   $Above_{Acc_{TH}} \leftarrow$  Bins with accuracy  $\geq Acc_{TH}$ 
12  for  $Below_{Acc_{TH}}$  and  $Above_{Acc_{TH}}$  do
13     $N_{B_{TH}} = \text{len}(Below_{Acc_{TH}})$ 
14     $N_{A_{TH}} = \text{len}(Above_{Acc_{TH}})$ 
15  end
16  /* Calculate samples per  $N_{Bin_i}$  */
17   $N_{norm} = w_1 \times N_{B_{TH}} + w_2 \times N_{A_{TH}}$ 
18   $N_{Below_{Acc_{TH}}} \leftarrow N_{Step} \times \frac{w_1}{N_{norm}}$ 
19   $N_{Above_{Acc_{TH}}} \leftarrow N_{Step} \times \frac{w_2}{N_{norm}}$ 
20   $N_{Bins} \leftarrow \lceil N_{Below_{Acc_{TH}}} \rceil + \lceil N_{Above_{Acc_{TH}}} \rceil$ 
21  for  $i$  in  $N_{Bin_i}$  do
22    Sample  $N_{Bin_i}$  from  $SS$ 
23    Append each sampled  $N_{Bin_i}$  to  $SS_{weighted}$ 
24  end
25  Append  $SS_{weighted}$  samples into  $SS_I$ 
26   $SS_{I,weighted} \leftarrow SS_I + SS_{weighted}$ 
27  return  $SS_{I,weighted}$ ;
28 end
```

E. Dataset Extension

If the predictor fails to meet the evaluation criteria, the framework looks to extend the dataset for re-training and re-evaluation of the predictor (see ④ in Fig. 5). This extension is executed using **Algo. 1**.

The user defines total number of additional training samples to select, N_{Step} . If the sampling strategy is *random*, N_{Step} samples are randomly selected from the entire sample space and appended to the initial sample space. These samples then proceed and undergo the usual dataset generation pipeline where first reference models are added, then the samples are encoded and executed on a target device. These new architecture-latency pairs are appended to the previous dataset D . The MLP is then re-trained and re-evaluated. This loop repeats until all the bins' accuracies, Acc_i , meet the threshold accuracy, Acc_{TH} .

If the user had defined the sampling strategy as *balanced*, a *weighted* approach with regards to the bins is taken. The bins, N_{Bins} are separated into two categories: Below Accuracy Threshold ($Below_{Acc_{TH}}$), and Above Accuracy Threshold

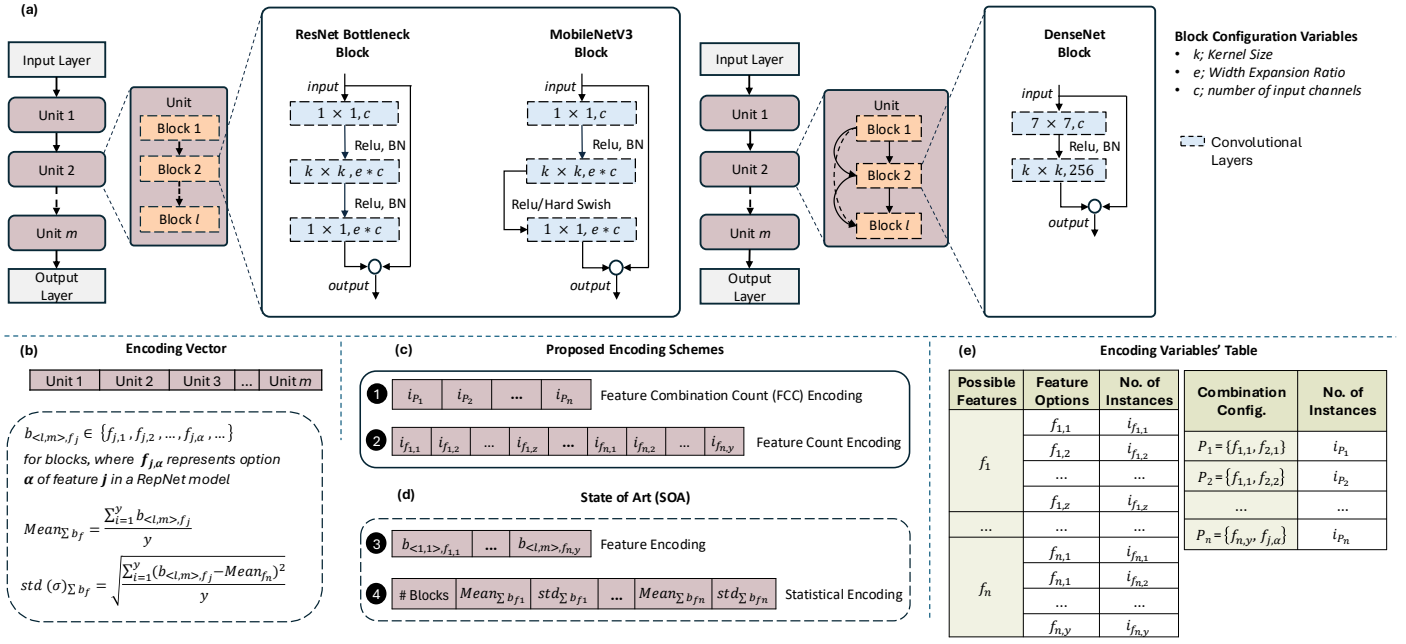


Fig. 7. (a) General representation of ResNet, MobileNetV3, and DenseNet supernets. (b) General representation of encoding, illustrating concatenation of unit-level features. (c) Our proposed encoding schemes. (d) State-of-the-art encoding schemes (e) Table containing information of variables used in the encoding schemes.

TABLE I
SUPERNET ARCHITECTURES AND ASSOCIATED HYPERPARAMETERS

Variable	ResNet	MobileNetV3	DenseNet
Stage Width List	$[2^8, 2^9, 2^{10}, 2^{11}]$	$[2^4, 2^5, 2^6, 2^7]$	N/A
# of Units	4	4	5
# of Blocks per Unit.	$\{1, 2, \dots, 7\}$	$\{1, 2, \dots, 7\}$	$\{1, 2, \dots, 20\}$
Kernel Size Options	$\{3, 5, 7\}$	$\{3, 5, 7\}$	$\{1, 3, 5, 7, 9\}^*$
Width-Expansion Ratio Options	$\{1/2, 2/3, 1\}$	$\{1/2, 2/3, 1\}$	N/A
# of Archs.	8.38×10^{26}	8.38×10^{26}	10^{10}

* For DenseNet, each kernel size is applied to all the blocks in a unit.

(Above_{Acc_{TH}}). For each category, number of samples are calculated by applying weights w_1 and w_2 , respectively, to N_{Step} for each category of bins. This can be seen in **Algo. 1**. This creates a bias in the dataset towards the below accuracy threshold. When re-training, the MLP learns their patterns as well and eventually crosses the accuracy threshold, Acc_{TH} .

III. RESULTS AND DISCUSSION

A. Experimental Setup

In this section, we describe the supernet architectures, regression models, and target hardware devices used in our experiments. We used three different supernet architectures: ResNet, MobileNetV3, and DenseNet. The associated hyperparameters are presented in Table I.

Regression Model: We used an MLP-based regression model consisting of three fully-connected layers, each with a hidden dimension size of 64. The model is trained using Mean Squared Error (MSE) as the loss function and the Adam optimizer, with a learning rate of 0.01 and a weight decay factor of 10^{-4} . Unless mentioned otherwise, the predictor is trained on 8,000 samples and tested on 4,000 samples.

Target Hardware: We evaluated our framework on multiple devices presented below:

- **CPUs:** AMD Ryzen Threadripper 5975, Raspberry Pi 4

- **GPUs:** NVIDIA RTX 4090, NVIDIA RTX 3080 Max-Q

B. Effectiveness of Proposed Encoding vs SoTA

Fig. 8 shows scatter plots of predicted latencies against actual latencies for ResNet and DenseNet. For this experiment, we trained the latency predictor on 8,000 and 20,000 samples using different encoding schemes for ResNet and DenseNet supernets. The plots show a comparison of the latency predictor's performance using different encoding schemes (proposed FCC vs. Statistical), and a different (Look-up Table-based) surrogate model. These results are further summarized in Fig. 9 where it can be seen that the MLP trained on FCC encoding comprehensively outperforms Look-up Table and Statistical Encoding methods. The predictor trained for ResNet supernet on FCC encoded data gave average accuracies of 97.6%, and 97.8% for 8,000 and 20,000 samples, respectively. Conversely, statistical encoding gave average accuracies of 85.8% and 83.1% for 8,000 and 20,000 samples, respectively. Performing bias correction using a linear regression model significantly improved the Lookup Table predictions with an average accuracy of 83.9% for ResNets. Similarly, FCC gave 99% accuracy, while Lookup Table with Bias Correction gave 97% accuracy for DenseNet.

C. FCC Encoding Effectiveness Across Devices

Figure 10 displays the performance of the predictor on the different target devices for the DNNs in Fig. 7(a). Again, for each device we present a comparison of our proposed FCC and FC encodings with SoTA Statistical Encoding. Training size samples for RTX 4090 was 8,000, 5,000 for AMD CPU and RTX 3080Ti, and 1,200 for Raspberry Pi 4. It can be seen that for each of the three DNNs, DenseNet, MobileNetV3, and ResNet, FCC and FC outperform Statistical Encoding on most devices. For ResNet super-net(Fig 10(a)), the average accuracies across the 4 devices listed were 97%, 88%, 93%, and 99% for proposed FCC encoding and 90%, 84%, 82% and 99% for FC encoding. While the averages for statistical encoding were 85%,

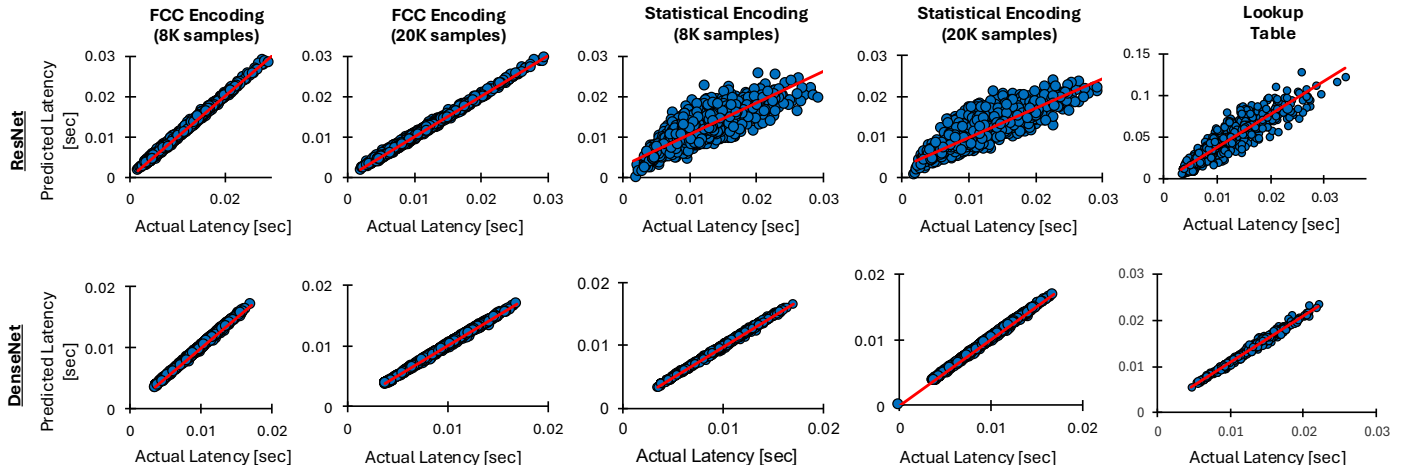


Fig. 8. Scatter plots illustrating a comparison between FCC-based MLP with Statistical Encoding-based MLP and Lookup-table for ResNet(Top-row) and DenseNet(Bottom-row). These results are for NVIDIA RTX 4090.

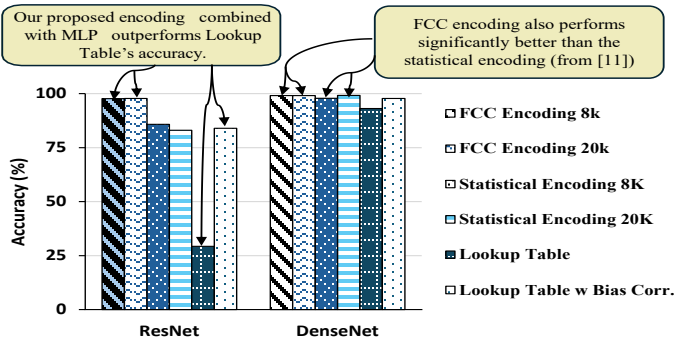


Fig. 9. Average accuracies' comparison for the cases presented in Fig 8.

83%, 71%, and 98%, respectively. For MobileNetV3 super-net (Fig. 10(b), the average accuracies across the 3 devices listed were 97%, 99%, and 99% for proposed FCC encoding, and 98%, 99% and 99% for FC encoding, while the averages for statistical encoding were 96%, 97%, and 97%, respectively. For DenseNet super-net (Fig. 10(c), the average accuracies across the 3 devices listed were 99%, 94%, and 94% for proposed FCC encoding and 99%, 96% and 95% for FC encoding, while the averages for statistical encoding were 91%, 96%, and 95%, respectively.

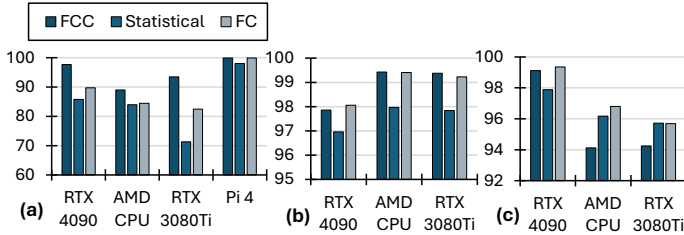


Fig. 10. Showcasing effectiveness of the proposed encoding schemes across different target devices: NVIDIA RTX 4090, AMD Ryzen Threadripper 5975, NVIDIA RTX 3080 Max-Q, and Raspberry Pi 4. The three plots are for (a) ResNet, (b) MobileNetV3, and (c) DenseNet.

D. Random and Balanced Accuracy vs Training Size

Figure 11 shows latency prediction results for *random* and *balanced* sampling strategies. For this experiment we computed latency of a ResNet super-net on NVIDIA RTX 4090. We

defined N_I as 300 and set N_{Step} to be 100 and used ESM methodology for training the predictor. The plot clearly shows that *balanced* sampling strategy converges much more earlier than *random* sampling strategy. *Balanced* sampling strategy converged after 3 iterations with 500 samples, while *random* sampling strategy took 37 iteration to converge with 4000 samples.

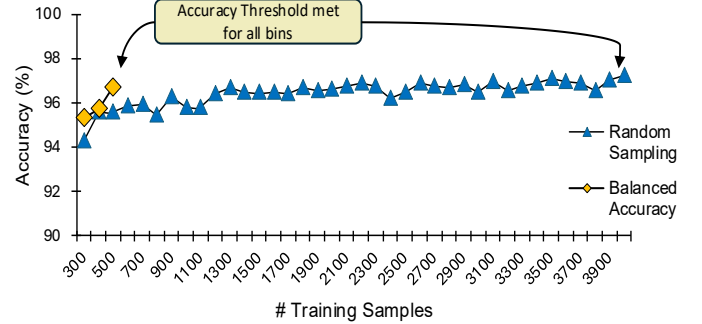


Fig. 11. Comparison between Random and Balanced Sampling Strategy for Latency Predictor tested on ResNet super-net on NVIDIA RTX 4090.

IV. CONCLUSION

In conclusion, this paper highlights the strengths and weaknesses of various surrogate models along with different encoding schemes. Our proposed framework, ESM, along with our novel encoding scheme, FCC, produce results that outperform current State-of-the-Art surrogate models and encoding schemes. Moreover, we provide clear analysis of how the ESM is scalable to a wide array of devices such as GPUs, CPUs, and embedded devices. Additionally, we also present how FCC can be scaled to various DNNs. Based on the insights gained from the analysis, we present a holistic framework that enables accurate, efficient latency prediction using HW-NAS for a variety of target hardware and neural networks, while adhering to user-defined constraints.

ACKNOWLEDGMENT

This work was partially supported by the NYUAD Center for Artificial Intelligence and Robotics (CAIR), funded by Tamkeen under the NYUAD Research Institute Award CG010.

REFERENCES

- [1] A. Voulodimos, N. Doulamis, A. Doulamis, and E. Protopapadakis, "Deep learning for computer vision: A brief review," *Computational intelligence and neuroscience*, vol. 2018, no. 1, p. 7068349, 2018.
- [2] D. W. Otter, J. R. Medina, and J. K. Kalita, "A survey of the usages of deep learning for natural language processing," *IEEE transactions on neural networks and learning systems*, vol. 32, no. 2, pp. 604–624, 2020.
- [3] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong *et al.*, "A survey of large language models," *arXiv preprint arXiv:2303.18223*, 2023.
- [4] Y. Chen, Y. Xie, L. Song, F. Chen, and T. Tang, "A survey of accelerator architectures for deep neural networks," *Engineering*, vol. 6, no. 3, pp. 264–274, 2020.
- [5] H. Cai, C. Gan, T. Wang, Z. Zhang, and S. Han, "Once-for-all: Train one network and specialize it for efficient deployment," *arXiv preprint arXiv:1908.09791*, 2019.
- [6] H. Benmeziane, K. El Maghraoui, H. Ouarnoughi, S. Niar, M. Wistuba, and N. Wang, "Hardware-aware neural architecture search: Survey and taxonomy," in *IJCAI*, 2021, pp. 4322–4329.
- [7] T.-J. Yang, A. Howard, B. Chen, X. Zhang, A. Go, M. Sandler, V. Sze, and H. Adam, "Netadapt: Platform-aware neural network adaptation for mobile applications," in *Proceedings of the European conference on computer vision (ECCV)*, 2018, pp. 285–300.
- [8] H. Cai, L. Zhu, and S. Han, "Proxylessnas: Direct neural architecture search on target task and hardware," *arXiv preprint arXiv:1812.00332*, 2018.
- [9] B. Wu, X. Dai, P. Zhang, Y. Wang, F. Sun, Y. Wu, Y. Tian, P. Vajda, Y. Jia, and K. Keutzer, "Fbnet: Hardware-aware efficient convnet design via differentiable neural architecture search," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2019, pp. 10 734–10 742.
- [10] T. Wang, K. Wang, H. Cai, J. Lin, Z. Liu, H. Wang, Y. Lin, and S. Han, "Apq: Joint search for network architecture, pruning and quantization policy," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 2078–2087.
- [11] H. Wang, Z. Wu, Z. Liu, H. Cai, L. Zhu, C. Gan, and S. Han, "Hat: Hardware-aware transformers for efficient natural language processing," *arXiv preprint arXiv:2005.14187*, 2020.
- [12] M. Tan and Q. Le, "Efficientnet: Rethinking model scaling for convolutional neural networks," in *International conference on machine learning*. PMLR, 2019, pp. 6105–6114.
- [13] X. Zhang, X. Zhou, M. Lin, and J. Sun, "Shufflenet: An extremely efficient convolutional neural network for mobile devices," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 6848–6856.
- [14] L. Mauch, S. Tiedemann, J. A. Garcia, B. N. Cong, K. Yoshiyama, F. Cardinaux, and T. Kemp, "Efficient sampling for predictor-based neural architecture search," *arXiv preprint arXiv:2011.12043*, 2020.
- [15] X. Dai, A. Wan, P. Zhang, B. Wu, Z. He, Z. Wei, K. Chen, Y. Tian, M. Yu, P. Vajda *et al.*, "Fbnetv3: Joint architecture-recipe search using predictor pretraining," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 16 276–16 285.
- [16] S. Hassantabar, X. Dai, and N. K. Jha, "Curious: Efficient neural architecture search based on a performance predictor and evolutionary search," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 11, pp. 4975–4990, 2022.
- [17] Z. Li, M. Paolieri, and L. Golubchik, "Inference latency prediction at the edge," *arXiv preprint arXiv:2210.02620*, 2022.
- [18] D. M. Mendoza and S. Wang, "Predicting latency of neural network inference," 2020.
- [19] X. Ning, Y. Zheng, T. Zhao, Y. Wang, and H. Yang, "A generic graph-based neural architecture encoding scheme for predictor-based nas," in *European Conference on Computer Vision*. Springer, 2020, pp. 189–204.